A motivation, scope, and plan for a physical quantities and units library

Document #:	P2980R0
Date:	2023-10-15
Project:	Programming Language C++
Audience:	Library Evolution Working Group

Reply-to:	Mateusz Pusz (Epam Systems) <mateusz.pusz@gmail.com> Dominik Berner <dominik.berner@gmail.com> Johel Ernesto Guerrero Peña <johelegp@gmail.com> Chip Hogg (Aurora Innovation) <charles.r.hogg@gmail.com> Nicolas Holthaus <nholthaus@gmail.com> Roth Michaels (Native Instruments) <isocxx@rothmichaels.us> Vincent Reverdy <vince.rev@gmail.com>

Contents

1 Introduction

Several groups in the ISO C++ Committee reviewed the “P1935: A C++ Approach to Physical Units” [P1935R2] proposal in Belfast 2019 and Prague 2020. All those groups expressed interest in the potential standardization of such a library and encouraged further work. The authors also got valuable initial feedback that highly influenced the design of the V2 version of the [mp-units] library.

In the following years, the library’s authors scoped on getting more feedback from the production and design and developed version 2 of the [mp-units] library that resolves the issues raised by the users and Committee members. The features and interfaces of this version are close to being the best we can get with the current version of the C++ language standard.

This paper is authored by not only the [mp-units] library developers but also by the authors of other actively maintained similar libraries on the market and other active members of the C++ physical quantities and units community who have worked on this subject for many years. We join our forces to say with one voice that we deeply care about standardizing such features as a part of the C++ Standard Library. Based on our long and broad experience in the subject, we agree that the interfaces we will provide in the upcoming proposals are the best we can get today in the C++ language.

2 Terms and definitions

This document consistently uses the official metrology vocabulary defined in the [ISO/IEC Guide 99] and [JCGM 200:2012].

3 About authors

3.1 Dominik Berner

Dominik is a strong believer that the C++ language can provide very high safety guarantees when programming through strong typing; a type error caught during compilation saves hours of debugging. For the last 15 years, he has mainly coded in C++ and actively follows its evolution through the new standards.

When working on regulated projects at Med-Tech, there usually were very tight requirements on which data types were to be used for what, which turned out to be lists of primitives to be memorized by each developer. However, throughout his career, Dominik spent way too many hours debugging and fixing issues caused by these types being incorrect. In an attempt to bring a closer semantic meaning to these lists, he eventually wrote [SI library] as a side project.

While [SI library] provides many useful features, such as type-safe conversion between physical quantities as well as zero-overhead computation for values of the same units,

there are some shortcomings which would require major rework. Instead of creating yet another library, Dominik decided to join forces with the other authors of this paper to push for standardizing support for more type-safety for physical quantities. He hopes that this will eventually lead to a safer and more robust C++ and open many more opportunities for the language.

§ 3.2 Johel Ernesto Guerrero Peña

Johel got interested in the units domain while writing his first hundred lines of game development. He got up to opening the game window, so this milestone was not reached until years later. Instead, he looked for the missing piece of abstraction, called “pixel” in the GUI framework, but modeled as an `int`. He found out about [nholthaus/units], and got fascinated with the idea of a library that succinctly allows expressing his domain’s units (https://github.com/nholthaus/units/issues/124#issuecomment-390773279).

Johel became a contributor to [nholthaus/units] v3 from 2018 to 2020. He improved the interfaces and implementations by remodeling them after `std::chrono::duration`. This included parameterizing the representation type with a template parameter instead of a macro. He also improved the error messages by mapping a list of types to an user-defined name.

By 2020, Johel had been aware of [mp-units] v0 `quantity<dim_length, length, int>`, put off by its verbosity. But then, he watched a talk by Mateusz Pusz on [mp-units]. It described how good error messages was a stake in the ground for the library. Thanks to his experience in the domain, Johel was convinced that [mp-units] was the future.

Since 2020, Johel has been contributing to [mp-units]. He added `quantity_point`, the generalization of `std::chrono::time_point`, closing #1. He also added `quantity_kind`, which explored the need of representing distinct quantities of the same dimension. To help guide its evolution, he’s been constantly pointing in the direction of [JCGM 200:2012] as a source of truth. And more recently, to the ISO/IEC 80000 series, also helping interpret it.

Computing systems engineer. (C++) programmer since 2014. Lives at HEAD with C++Next and good practices. Performs in-depth code reviews of familiarized code bases. Has an eye for identifying automation opportunities, and acts on them. Mostly at https://github.com/JohelEGP/.

§ 3.3 Charles Hogg

Chip Hogg is a Staff Software Engineer on the Motion Planning Team at Aurora Innovation, the self-driving vehicle company that is developing the Aurora Driver. After obtaining his PhD in Physics from Carnegie Mellon in 2010, he was a postdoctoral researcher and then staff scientist at the National Institute of Standards and Technology (NIST), doing Bayesian data analysis. He joined Google in 2012 as a software engineer, leaving in 2016 to work on autonomous vehicles at Uber’s Advanced Technologies Group (ATG), where he stayed until their acquisition by Aurora in 2021.

Chip built his first C++ units library at Uber ATG in 2018, where he first developed the concept of unit-safe interfaces. At Aurora in 2021, he ported over only the test cases, writing a new and more powerful units library from scratch. This included novel features such as vector space magnitudes, and an adaptive conversion policy which guards against overflow in integers.

He soon realized that there was a much broader need for Aurora's units library. No publicly available units library for C++14 or C++17 could match its ergonomics, developer experience, and performance. This motivated him to create [Au] in 2022: a new, zero-dependency units library, which was a drop-in replacement for Aurora's original units library, but offered far more composable interfaces, and was built on simpler, stronger foundations. Once Au proved its value internally, Chip migrated it to a separate repository and led the open-sourcing process, culminating in its public release in 2023.

While Au provides excellent ergonomics and robustness for pre-C++20 users, Chip also believes the C++ community would benefit from a standard units library. For that reason, he has joined forces with the mp-units project, contributing code and design ideas.

§ 3.4 Nicolas Holthaus

Nicolas graduated Summa Cum Laude from Northwestern University with a B.S. in Computer Engineering. He worked for several years at the United States Naval Air Warfare Center - Manned Flight Simulator - designing real-time C++ software for aircraft survivability simulation. He has subsequently continued in the field at various start-ups, MIT Lincoln Laboratory, and most recently, STR (Science and Technology Research).

Nicolas became obsessed with dimensional analysis as a high school JETS team member after learning that the $125M Mars Climate Orbiter was destroyed due to a simple feet-to-meters miscalculation. He developed the widely adopted C++ [nholthaus/units] library based on the findings of the 2002 white paper "Dimensional Analysis in C++" by Scott Meyers. Astounded that no one smarter had already written such a library, he continued with `units` 2.0 and 3.0 based on modern C++. Those libraries have been extensively adopted in many fields, including modeling & simulation, agriculture, and geodesy.

In 2023, recognizing the limits of `units`, he joined forces with Mateusz Pusz in his effort to standardize his evolutionary dimensional analysis library, with the goal of providing the highest-quality dimensional analysis to all C++ users via the C++ standard library.

§ 3.5 Roth Michaels

Roth Michaels is a Principal Software Engineer at Native Instruments, a leading manufacturer of audio, and music, software and hardware. Working in this domain, he has been involved with the creation of ad hoc typed quantities/units for digital signal processing and GUI library use-cases. Seeing both the complexity of development and practical uses where developers need to leave the safety of these simple wrappers encouraged Roth to explore various quantity/units libraries to see if they would apply to

this domain. He has been doing research into defining and using digital audio and music domain-specific quantities and units using first [mp-units] as proposed in [P1935R2] and the new V2 library described in this paper.

Before working for Native Instruments, Roth worked as a consultant in multiple industries using a variety of programming languages. He was involved with the Swift Evolution community in its early days before focusing primarily on C++ after joining iZotope and now Native Instruments.

Holding a degree in music composition, Roth has over a decade of experience working with quantities and units of measure related to music, digital signal processing, analog audio, and acoustics. He has joined the [mp-units] project as a domain expert in these areas and to provide perspective on logarithmic and non-linear quantity/unit relationships.

§ 3.6 Mateusz Pusz

Mateusz got interested in the physical units subject while contributing to the [LK8000] Tactical Flight Computer Open Source project over 10 years ago. The project's code was far from being "safe" in the C++ sense, and this is when Mateusz started to explore alternatives.

Through the following years, he tried to use several existing solutions, which were always far from being user-friendly, so he also tried to write a better framework a few times from scratch by himself.

Finally, with the availability of brand new Concepts TS in the gcc-7, the [mp-units] project was created. It was designed with safety and user experience in mind. After many years of working on the project, the [mp-units] library is probably the most modern and complete solution in the C++ market.

Through the last few years, Mateusz has put much effort into building a community around physical units. He provided many talks and workshops on this subject at various C++ conferences. He also approached the authors of other actively maintained libraries to get their feedback and invited them to work together to find and agree on the best solution for the C++ language. This paper is the result of those actions.

§ 3.7 Vincent Reverdy

Vincent is an astrophysicist, computer scientist, and a member of the French delegation to the ISO C++ Committee, currently working as a full researcher at the French National Centre for Scientific Research (CNRS). He has been interested for years in units and quantities for programming languages to ensure higher levels of both expressivity and safety in computational physics codes. Back in 2019, he authored [P1930R0] to provide some context of what could be a quantity and unit library for C++.

After designing and implementing several Domain-Specific Language (DSL) demonstrators dedicated to units of measurements in C++, he became more interested in

the theoretical side of the problem. Today, one of his research activities is dedicated to the mathematical formalization of systems of quantities and systems of units as an interdisciplinary problem between physics, mathematics, and computer science.

§ 4 Motivation

This chapter describes why we believe that physical quantities and units should be part of a C++ Standard Library.

§ 4.1 Safety

It is no longer only the space industry or experienced pilots that benefit from the autonomous operations of some machines. We live in a world where more and more ordinary people trust machines with their lives daily. In the near future, we will be allowed to sleep while our car autonomously drives us home from a late party. As a result, many more C++ engineers are expected to write life-critical software today than it was a few years ago. However, writing safety-critical code requires extensive training and experience, both of which are in short demand. While there exists some standards and guidelines such as MISRA C++ [MISRA C++] with the aim of enforcing the creation of safe code in C++, they are cumbersome to use and tend to shift the burden on the discipline of the programmers to enforce these. At the time of writing, the C++ language does not change fast enough to enforce safe-by-construction code.

One of the ways C++ can significantly improve the safety of applications being written by thousands of developers is by introducing a type-safe, well-tested, standardized way to handle physical quantities and their units. The rationale is that people tend to have problems communicating or using proper units in code and daily life. Numerous expensive failures and accidents happened due to using an invalid unit or a quantity type.

The most famous and probably the most expensive example in the software engineering domain is the Mars Climate Orbiter that in 1999 failed to enter Mars' orbit and crashed while entering its atmosphere [Mars Orbiter]. This is one of many examples here. People tend to confuse units quite often. We see similar errors occurring in various domains over the years:

— On October 12, 1492, Christopher Columbus unintentionally discovered the sea route from Europe to America because, during his travel preparations, he mixed the Arabic mile with a Roman mile, which led to the wrong estimation of the equator and his expected travel distance [Columbus].

— In 1628, a new warship, Vasa, accidentally had an asymmetrical hull (being thicker on the port side than the starboard side), which was one of the reasons for her sinking less than a mile into her maiden voyage, resulting in the death of 30 people on board. This asymmetry could have been caused by the use of different systems of measurement, as archaeologists have found four rulers used by the workers who built

the ship. Two were calibrated in Swedish feet, which had 12 inches, while the other two measured Amsterdam feet, which had 11 inches [Vasa].

— Air Canada Flight 143 ran out of fuel on July 23, 1983, at an altitude of 41 000 feet (12 000 metres), midway through the flight because the fuel had been calculated in pounds instead of kilograms by the ground crew [Gimli Glider].

— The British rock band Black Sabbath, during its Born Again tour in 1983, ordered a replica of Stonehenge as props for the scene. Unfortunately, they had to leave them in the storage area because, while submitting the order, their manager wrote dimensions down in meters when he meant feet, and so the stones didn't fit the scene. "It cost a fortune to make, but there was not a building on Earth that you could fit it into" [Stonehenge].

— On April 15, 1999, Korean Air Cargo Flight 6316 crashed due to a miscommunication between pilots about the desired flight altitude [Flight 6316].

— In February 2001, the crew of the Moorpark College Zoo built an enclosure for Clarence the Tortoise with a weight of 250 pounds instead of 250 kilograms [Clarence].

— In December 2003, one of the roller coaster's cars at Tokyo Disneyland's Space Mountain attraction suddenly derailed due to a broken axle caused by confusion after upgrading the specification from imperial to metric units [Disney].

— During the construction of the Hochrheinbrücke bridge to connect the small German town of Laufenburg with Swiss Laufenburg, the construction team made a sign error that resulted in a discrepancy of 54 cm between the two outer ends of the bridge [Hochrheinbrücke].

— An American company sold a shipment of wild rice to a Japanese customer, quoting a price of 39 cents per pound, but the customer thought the quote was for 39 cents per kilogram [Wild Rice].

— A whole set of [Medication dose errors]…

The safety subject is so vast and essential by itself that we dedicated a separate paper [P2981] that discusses all the nuances in detail.

§ 4.2 Vocabulary types

We standardized many library features mostly used in the implementation details (fmt, ranges, random-number generators, etc.). However, we believe that the most important role of the C++ Standard is to provide a standardized way of communication between different vendors.

Let's imagine a world without `std::string` or `std::vector`. Every vendor has their version of it, and of course, they are highly incompatible with each other. As a result, when someone needs to integrate software from different vendors, it turns out to be an unnecessarily arduous task.

Introducing `std::chrono::duration` and `std::chrono::time_point` improved the interfaces a lot, but time is only one of many quantities that we deal with in our software on a daily basis. We desperately need to be able to express more quantities and units in a standardized way so different libraries get means to communicate with each other.

If Lockheed Martin and NASA could have used standardized vocabulary types in their interfaces, maybe they would not interpret pound-force seconds as newton seconds, and the [Mars Orbiter] would not have crashed during the Mars orbital insertion maneuver.

§ 4.3 Certification

Mission and life-critical projects, or those for embedded devices, often have to obey the safety norms that care about software for safety-critical systems (e.g., ISO 61508 is a basic functional safety standard applicable to all industries, and ISO 26262 for automotive). As a result, their company policy often forbid third-party tooling that lacks official certification. Such certification requires a specification to be certified against, and those tools often do not have one. The risk and cost of self-certifying an Open Source project is too high for many as well.

Companies often have a policy that the software they use must obey all the rules MISRA provides. This is a common misconception, as many of those rules are intended to be deviated from. However, those deviations require rationale and documentation, which is also considered to be risky and expensive by many.

All of those reasons often prevent the usage of an Open Source product in a company, which is a huge issue, as those companies typically are natural users of physical quantities and units libraries.

Having the physical quantities and units library standardized would solve those issues for many customers, and would allow them to produce safer code for projects on which human life depends every single day.

§ 4.4 Complex and complicated

Suppose vendors can’t use an Open Source library in a production project for the above reasons. They are forced to write their own abstractions by themselves. Besides being costly and time-consuming, it also happens that writing a physical quantities and units library by yourself is far from easy. Doing this is complex and complicated, especially for engineers who are not experts in the domain. There are many exceptional corner cases to cover that most developers do not even realize before falling into a trap in production. On the other hand, domain experts might find it difficult to put their knowledge into code and create a correct implementation in C++. As a result, companies either use really simple and unsafe numeric wrappers, or abandon the effort entirely and just use built-in types, such as `float` or `int`, to express quantity values, thus losing all semantic categorization. This often leads to safety issues caused by accidentally using values representing the wrong quantity or having an incorrect unit.

§ 4.5 Extensibility

Many applications of a quantity and units library may need to operate on a combination of standard (e.g. SI) and domain-specific quantities and units. The complexity of developing domain-specific solutions highlights the value in being able to define new quantities and units that have all the expressivity and safety as those provided by the library.

Experience with writing ad hoc typed quantities without library support that can be combined with or converted to `std::chrono::duration` has shown the downside of bespoke solutions: If not all operations or conversions are handled, users will need to leave the safety of typed quantities to operate on primitive types.

The interfaces of the [mp-units] library were designed with ease of extensibility in mind. Each definition of a dimension, quantity type, or unit typically takes only a single line of code. This is possible thanks to the extensive usage of C++20 class types as Non-Type Template Parameters (NTTP). For example, the following code presents how second (a unit of time in the [SI]) and hertz (a unit of frequency in the [SI]) can be defined:

```
inline constexpr struct second : named_unit<"s", kind_of<isq::time>> {}
        second;
inline constexpr struct hertz : named_unit<"Hz", 1 / second,
        kind_of<isq::frequency>> {} hertz;
```

§ 4.6 Broad industry value

When people think about industries that could use physical quantities and unit libraries, they think of a few companies related to aerospace, autonomous cars, or embedded industries. That is all true, but there are many other potential users for such a library.

Here is a list of some less obvious candidates:

— Manufacturing,

— maritime industry,

— freight transport,

— military,

— astronomy,

— 3D design,

— robotics,

— audio,

— medical devices,

— national laboratories,

— scientific institutions and universities,

— all kinds of navigation and charting,

— GUI frameworks,

— finance (including HFT).

As we can see, the range of domains for such a library is vast and not limited to applications involving specifically physical units. Any software that involves measurements, or operations on counts of some standard or domain-specific quantities, could benefit from a zero-cost abstraction for operating on quantity values and their units. The library also provides affine space abstractions, which may prove useful in many applications.

§ 4.7 Standardizing existing practice

Plenty of physical units libraries have been available to the public for many years. Throughout the years, we have learned the best practices for handling specific cases in the domain. Various products may have different scopes and support different C++ versions. Still, taking that aside, they use really similar concepts, types, and operations under the hood. We know how to do those things already.

The authors of this paper developed and delivered multiple successful C++ libraries for this domain. They joined forces and are working together to propose the best physical quantities and units library we can get with the latest version of the C++ language. They spend their private time and efforts hoping that the ISO C++ Committee will be willing to include such a feature in the C++ standard library.

§ 5 Design goals

The library facilities that we plan to propose in the upcoming papers will be designed with the following goals in mind.

§ 5.1 Compile-time safety

The most important property of such a library is the safety it brings to C++ projects. The correct handling of physical quantities, units, and numerical values should be verified both by the compiler during the compilation process and by humans with manual inspection in each individual line.

In some cases we are even eager to prioritize safe interfaces over the general usability experience (e.g. getters of the underlying raw numerical value will always require a unit in which the value should be returned in, which results in more typing and is sometimes redundant).

More information on this subject can be found in [P2981].

§ 5.2 Performance

The library should be as fast or even faster than working with fundamental types. The should be no runtime overhead and no space size overhead should be needed to implement high-level abstractions.

§ 5.3 Great user experience

The primary purpose of such a library is to generate compile-time errors. If the developers did not introduce any bugs in the manual handling of quantities and units, such a library would be of little use. This is why such a library should be optimized for readable compilation errors and great debugging experience.

Such a library should be also easy to use and flexible. The interfaces should be straightforward and safe by default. Users should be able to easily express any quantity and unit, which requires them to compose.

The above constraints imply the usage of special implementation techniques. The library will not only provides types but also compile-time known values that will enable users to write easy to understand and efficient equations on quantities and units.

§ 5.4 Feature rich

There are plenty of expectations from different parties regarding such a library. It should support at least:

— Systems of Quantities,

— Systems of Units,

— Scalar, vector, and tensor quantities,

— The affine space,

— Natural units systems,

— Strong angular system,

— Any unit’s magnitude (huge, small, floating-point),

— Faster-than-lightspeed constants,

— Highly adjustable text-output formatting.

§ 5.5 Easy to extend

Most entities in the library should be possible to be defined with a single line of code with no need to use preprocessor macros. Users should be able to easily extend provided systems with custom dimensions, quantities, and units.

§ 5.6 Low standardization cost

The set of entities required for standardization should be limited to the bare minimum.

Derived units should not require separate library types but should be obtained through the composition of predefined named units. Units should not be associated with User-Defined Literals (UDLs), as it is the case of `std::chrono::duration`. UDLs do not compose, have very limited scope and functionality, and are expensive to standardize.

There should be no preprocessor macros in the user interface.

Most proposed features (besides text output) should be possible to be standardized as a freestanding part of the C++ standard library.

§ 6 Look and feel

Note: The code examples presented in this paper may not exactly reflect the final interface design that is going to be proposed in the follow-up papers. We are still doing some small fine-tuning to improve the library.

So far, no papers have been submitted on a working interface of physical quantities and units. To allow the reader to better understand the features and scope of the library, this chapter presents a few simple examples.

§ 6.1 Basic quantity equations

Let's start with a really simple example presenting basic operations that every physical quantities and units library should provide:

```
#include <mp-units/systems/si/si.h>

using namespace mp_units;
using namespace mp_units::si::unit_symbols;

// simple numeric operations
static_assert(10 * km / 2 == 5 * km);

// unit conversions
static_assert(1 * h == 3600 * s);
static_assert(1 * km + 1 * m == 1001 * m);

// derived quantities
static_assert(1 * km / (1 * s) == 1000 * m / s);
static_assert(2 * km / h * (2 * h) == 4 * km);
static_assert(2 * km / (2 * km / h) == 1 * h);

static_assert(2 * m * (3 * m) == 6 * m2);

static_assert(10 * km / (5 * km) == 2 * one);

static_assert(1000 / (1 * s) == 1 * kHz);
```

Try it in the Compiler Explorer.

§ 6.2 Hello units

The next example serves as a showcase of various features available in the [mp-units] library.

```
#include <mp-units/format.h>
#include <mp-units/ostream.h>
#include <mp-units/systems/international/international.h>
#include <mp-units/systems/isq/isq.h>
#include <mp-units/systems/si/si.h>
#include <iostream>

using namespace mp_units;

constexpr QuantityOf<isq::speed> auto avg_speed(QuantityOf<isq::length>
        auto d,
                                                QuantityOf<isq::time> auto
        t)
{
  return d / t;
}

int main()
{
  using namespace mp_units::si::unit_symbols;
  using namespace mp_units::international::unit_symbols;

  constexpr quantity v1 = 110 * km / h;
  constexpr quantity v2 = 70 * mph;
  constexpr quantity v3 = avg_speed(220. * km, 2 * h);
    constexpr quantity v4 = avg_speed(isq::distance(140. * mi), 2 *
        isq::duration[h]);
  constexpr quantity v5 = v3.in(m / s);
  constexpr quantity v6 = value_cast<m / s>(v4);
  constexpr quantity v7 = value_cast<int>(v6);

  std::cout << v1 << '\n';                // 110 km/h
  std::cout << v2 << '\n';                // 70 mi/h
  std::println("{}", v3);                 // 110 km/h
  std::println("{:*^14}", v4);            // ***70 mi/h****
  std::println("{:%Q in %q}", v5);        // 30.5556 in m/s
  std::println("{0:%Q} in {0:%q}", v6);   // 31.2928 in m/s
  std::println("{:%Q}", v7);              // 31
}
```

Try it in the Compiler Explorer.

§ 6.3 Bridge across the Rhine

The following example codifies the history of a famous issue during the construction of a bridge across the Rhine River between the German and Swiss parts of the town Laufenburg [Hochrheinbrücke]. It also nicely presents how the Affine Space is being modeled in the library.

```cpp
#include <mp-units/ostream.h>
#include <mp-units/quantity_point.h>
#include <mp-units/systems/isq/space_and_time.h>
#include <mp-units/systems/si/si.h>
#include <iostream>

using namespace mp_units;
using namespace mp_units::si::unit_symbols;

constexpr struct amsterdam_sea_level :
        absolute_point_origin<isq::altitude> {
} amsterdam_sea_level;

constexpr struct mediterranean_sea_level :
        relative_point_origin<amsterdam_sea_level + isq::altitude(-27 *
        cm)> {
} mediterranean_sea_level;

using altitude_DE = quantity_point<isq::altitude[m], amsterdam_sea_level>;
using altitude_CH = quantity_point<isq::altitude[m],
        mediterranean_sea_level>;

template<auto R, typename Rep>
std::ostream& operator<<(std::ostream& os, quantity_point<R,
        altitude_DE::point_origin, Rep> alt)
{
  return os << alt.quantity_ref_from(alt.point_origin) << " AMSL(DE)";
}

template<auto R, typename Rep>
std::ostream& operator<<(std::ostream& os, quantity_point<R,
        altitude_CH::point_origin, Rep> alt)
{
  return os << alt.quantity_ref_from(alt.point_origin) << " AMSL(CH)";
}

int main()
{
  // expected bridge altitude in a specific reference system
      quantity_point expected_bridge_alt = amsterdam_sea_level +
        isq::altitude(330 * m);

  // some nearest landmark altitudes on both sides of the river
  // equal but not equal ;-)
  altitude_DE landmark_alt_DE = altitude_DE::point_origin + 300 * m;
  altitude_CH landmark_alt_CH = altitude_CH::point_origin + 300 * m;

  // artifical deltas from landmarks of the bridge base on both sides of
        the river
  quantity delta_DE = isq::height(3 * m);
  quantity delta_CH = isq::height(-2 * m);

  // artificial altitude of the bridge base on both sides of the river
  quantity_point bridge_base_alt_DE = landmark_alt_DE + delta_DE;
  quantity_point bridge_base_alt_CH = landmark_alt_CH + delta_CH;
```

```
  // artificial height of the required bridge pilar height on both sides
       of the river
       quantity    bridge_pilar_height_DE    =    expected_bridge_alt    -
       bridge_base_alt_DE;
       quantity    bridge_pilar_height_CH    =    expected_bridge_alt    -
       bridge_base_alt_CH;

  std::cout << "Bridge pillars height:\n";
  std::cout << "- Germany:     " << bridge_pilar_height_DE << '\n';
  std::cout << "- Switzerland: " << bridge_pilar_height_CH << '\n';

  // artificial bridge altitude on both sides of the river in both systems
       quantity_point    bridge_road_alt_DE    =    bridge_base_alt_DE    +
       bridge_pilar_height_DE;
       quantity_point    bridge_road_alt_CH    =    bridge_base_alt_CH    +
       bridge_pilar_height_CH;

  std::cout << "Bridge road altitude:\n";
  std::cout << "- Germany:     " << bridge_road_alt_DE << '\n';
  std::cout << "- Switzerland: " << bridge_road_alt_CH << '\n';

    std::cout  <<  "Bridge  road  altitude  relative  to  the  Amsterdam  Sea
       Level:\n";
     std::cout   <<   "-   Germany:            "   <<   bridge_road_alt_DE   -
       amsterdam_sea_level << '\n';
      std::cout   <<   "-   Switzerland:   "   <<   bridge_road_alt_CH   -
       amsterdam_sea_level << '\n';
}
```

The above provides the following text output:

```
Bridge pillars height:
- Germany:     27 m
- Switzerland: 3227 cm
Bridge road altitude:
- Germany:     330 m AMSL(DE)
- Switzerland: 33027 cm AMSL(CH)
Bridge road altitude relative to the Amsterdam Sea Level:
- Germany:     330 m
- Switzerland: 33000 cm
```

Try it in the Compiler Explorer.

§ 6.4 Storage tank

This example estimates the process of filling a storage tank with some contents. It presents:

— The importance of supporting more than one distinct quantity of the same kind,

— faster-than-lightspeed constants,

— how easy it is to add custom quantity types when needed, and

— interoperability with `std::chrono::duration`.

```cpp
#include <mp-units/chrono.h>
#include <mp-units/format.h>
#include <mp-units/math.h>
#include <mp-units/systems/isq/isq.h>
#include <mp-units/systems/si/si.h>
#include <cassert>
#include <chrono>
#include <format>
#include <numbers>
#include <utility>

// allows standard gravity (acceleration) and weight (force) to be
    expressed with scalar representation
// types instead of requiring the usage of Linear Algebra library for this
    simple example
template<class T>
  requires mp_units::is_scalar<T>
inline constexpr bool mp_units::is_vector<T> = true;

namespace {

using namespace mp_units;
using namespace mp_units::si::unit_symbols;

// add a custom quantity type of kind isq::length
inline constexpr struct horizontal_length : quantity_spec<isq::length> {}
    horizontal_length;

// add a custom derived quantity type of kind isq::area with a constrained
    quantity equation
inline constexpr struct horizontal_area : quantity_spec<isq::area,
    horizontal_length * isq::width> {} horizontal_area;

inline constexpr auto g = 1 * si::standard_gravity;
inline constexpr auto air_density = isq::mass_density(1.225 * kg / m3);

class StorageTank {
  quantity<horizontal_area[m2]> base_;
  quantity<isq::height[m]> height_;
  quantity<isq::mass_density[kg / m3]> density_ = air_density;
public:
  constexpr StorageTank(const quantity<horizontal_area[m2]>& base, const
      quantity<isq::height[m]>& height) :
    base_(base), height_(height)
  {
  }

  constexpr void set_contents_density(const quantity<isq::mass_density[kg
      / m3]>& density)
  {
    assert(density > air_density);
    density_ = density;
  }

  [[nodiscard]] constexpr QuantityOf<isq::weight> auto filled_weight()
      const
  {
```

```
    const auto volume = isq::volume(base_ * height_);
    const QuantityOf<isq::mass> auto mass = density_ * volume;
    return isq::weight(mass * g);
  }

    [[nodiscard]]  constexpr  quantity<isq::height[m]>  fill_level(const
        quantity<isq::mass[kg]>& measured_mass) const
  {
    return height_ * measured_mass * g / filled_weight();
  }

   [[nodiscard]] constexpr quantity<isq::volume[m3]> spare_capacity(const
        quantity<isq::mass[kg]>& measured_mass) const
  {
    return (height_ - fill_level(measured_mass)) * base_;
  }
};


class CylindricalStorageTank : public StorageTank {
public:
  constexpr CylindricalStorageTank(const quantity<isq::radius[m]>& radius,
                                   const quantity<isq::height[m]>& height)
        :
      StorageTank(quantity_cast<horizontal_area>(std::numbers::pi * pow<2>
        (radius)), height)
  {
  }
};

class RectangularStorageTank : public StorageTank {
public:
   constexpr RectangularStorageTank(const quantity<horizontal_length[m]>&
        length,
                                    const quantity<isq::width[m]>& width,
                                    const quantity<isq::height[m]>& height)
        :
      StorageTank(length * width, height)
  {
  }
};

}  // namespace


int main()
{
  const auto height = isq::height(200 * mm);
    auto  tank  =  RectangularStorageTank(horizontal_length(1'000  *  mm),
        isq::width(500 * mm), height);
  tank.set_contents_density(1'000 * kg / m3);

  const auto duration = std::chrono::seconds{200};
  const quantity fill_time = value_cast<int>(quantity{duration});  // time
        since starting fill
  const quantity measured_mass = 20. * kg;                          //
        measured mass at fill_time
```

```
  const auto fill_level = tank.fill_level(measured_mass);
  const auto spare_capacity = tank.spare_capacity(measured_mass);
  const auto filled_weight = tank.filled_weight();

      const   QuantityOf<isq::mass_change_rate>   auto   input_flow_rate   =
          measured_mass / fill_time;
    const  QuantityOf<isq::speed>  auto  float_rise_rate  =  fill_level  /
          fill_time;
  const QuantityOf<isq::time> auto fill_time_left = (height / fill_level -
        1 * one) * fill_time;

  const auto fill_ratio = fill_level / height;

  std::println("fill height at {} = {} ({} full)", fill_time, fill_level,
        fill_ratio.in(percent));
   std::println("fill weight at {} = {} ({})", fill_time, filled_weight,
        filled_weight.in(N));
  std::println("spare capacity at {} = {}", fill_time, spare_capacity);
  std::println("input flow rate = {}", input_flow_rate);
  std::println("float rise rate = {}", float_rise_rate);
     std::println("tank  full  E.T.A.  at  current  flow  rate  =  {}",
        fill_time_left.in(s));
}
```

The above code outputs:

```
fill height at 200 s = 0.04 m (20 % full)
fill weight at 200 s = 100 g₀ kg (980.665 N)
spare capacity at 200 s = 0.08 m³
input flow rate = 0.1 kg/s
float rise rate = 0.0002 m/s
tank full E.T.A. at current flow rate = 800 s
```

Try it in the Compiler Explorer.

§ 6.5 User defined quantities and units

Users can easily define new quantities and units for domain-specific use-cases. This example from digital signal processing will show how to define custom units for counting digital samples and how they can be converted to time measured in milliseconds:

```
#include <mp-units/format.h>
#include <mp-units/systems/isq/isq.h>
#include <mp-units/systems/si/si.h>
#include <format>

namespace dsp_dsq {

using namespace mp_units;

inline   constexpr   struct   SampleCount   :   quantity_spec<dimensionless,
        is_kind> {} SampleCount;
inline  constexpr  struct  SampleDuration  :  quantity_spec<isq::time>  {}
        SampleDuration;
```

```
inline constexpr struct SamplingRate : quantity_spec<isq::frequency,
        SampleCount / isq::time> {} SamplingRate;

inline constexpr struct Sample : named_unit<"Smpl", one,
        kind_of<SampleCount>> {} Sample;

namespace unit_symbols {
inline constexpr auto Smpl = Sample;
}

}

int main()
{
  using namespace dsp_dsq::unit_symbols;
  using namespace mp_units::si::unit_symbols;

  const auto sr1 = 44100.f * Hz;
  const auto sr2 = 48000.f * Smpl / s;

  const auto bufferSize = 512 * Smpl;

  const auto sampleTime1 = (bufferSize / sr1).in(s);
  const auto sampleTime2 = (bufferSize / sr2).in(ms);

  const auto sampleDuration1 = (1 / sr1).in(ms);
  const auto sampleDuration2 = dsp_dsq::SampleDuration(1 / sr2).in(ms);

  const auto rampTime = 35.f * ms;
  const auto rampSamples1 = value_cast<int>((rampTime * sr1).in(Smpl));
  const auto rampSamples2 = value_cast<int>((rampTime * sr2).in(Smpl));

  std::println("Sample rate 1 is: {}", sr1);
  std::println("Sample rate 2 is: {}", sr2);

  std::println("{} @ {} is {}", bufferSize, sr1, sampleTime1);
  std::println("{} @ {} is {}", bufferSize, sr2, sampleTime2);

  std::println("One sample @ {} is {}", sr1, sampleDuration1);
  std::println("One sample @ {} is {}", sr2, sampleDuration2);

  std::println("{} is {} @ {}", rampTime, rampSamples1, sr1);
  std::println("{} is {} @ {}", rampTime, rampSamples2, sr2);
}
```

The above code outputs:

```
Sample rate 1 is: 44100 Hz
Sample rate 2 is: 48000 Smpl/s
512 Smpl @ 44100 Hz is 0.01161 s
512 Smpl @ 48000 Smpl/s is 10.6667 ms
One sample @ 44100 Hz is 0.0226757 ms
One sample @ 48000 Smpl/s is 0.0208333 ms
35 ms is 1543 Smpl @ 44100 Hz
35 ms is 1680 Smpl @ 48000 Smpl/s
```

Try it in the Compiler Explorer.

§ 7 Scope

The tables below briefly highlight the expected scope and feature set. Each of the features will be described in detail in the upcoming papers. To learn more right away and to be able to provide early feedback, we encourage everyone to check out the documentation of the [mp-units] project.

Note: The priorities provided in the below tables are the recommendations by authors based on their experience in the domain, but are in no way final. This is just an entry point for the discussion in the Committee.

§ 7.1 Basic framework

Feature	Priority	Description
Core library	1	`std::quantity`, expression templates, dimensions, quantity specifications, units, *references*, and concepts for them
Quantity kinds	1	Support quantities of the same dimension that should be distinct, e.g., `frequency`, `activity`, and `modulation_rate`, or `energy` and `moment_of_force`
Various quantities of the same kind	1	Support quantities of the same kind that should be distinct, e.g., `width`, `height`, `wavelength` (all of the kind `length`)
Vector and tensor representation types	2	Support for quantities of vector and tensor representation types (without intrusive changes on vector and tensor types)
Logarithmic units support	2	Support for logarithmic units, e.g., decibel
Polymorphic unit	???	Runtime-known type-erased unit of a specified quantity type

In the above table:

— `std::quantity` is a class template that is the workhorse of the library. It is an incompatible generalization of `std::chrono::duration`.

— Expression templates are used to provide user-friendly error messages and great debugging experience. More information on this subject can be found in [P2982].

— Quantity specifications include at least the following information: quantity type, quantity character, quantity equation (in case of derived quantities), quantity kind, and dimension.

— References (temporary name to be bikeshedded) are numeric wrappers over a quantity specification and an unit.

Below, we provide a short overview of estimated additional framework-related costs associated with providing support for the following features (based on our current implementation experience):

— Quantity kinds
  — `is_kind` tag type
  — additional conversion rules to improve safety
— Quantities of the same kind
  — `kind_of<QS>` class and variable templates
  — additional conversion rules to improve safety
  — `common_quantity_spec()` function that finds a common node in the hierarchy tree
— Vector and tensor representation types
  — `quantity_character` enum
  — `is_scalar<T>`, `is_vector<T>`, `is_tensor<T>` customizations points
  — a few concepts (e.g. `RepresentationOf<Ch>`, `VectorRepresntation<T>`, …)
— Logarithmic units
  — the design is TBD
— Natural units
  — `system_reference` class template
— Polymorphic unit
  — the design is TBD

§ 7.2 The affine space

Feature	Priority	Description
The Affine Space	1	`std::quantity_point`, absolute and relative point origins

In the above table:

— `std::quantity_point` is an incompatible generalization of `std::chrono::time_point`.

— Absolute point origin is a compile-time constant defining an absolute origin for `std::quantity_point` of a specific quantity type.

— Relative point origin is a compile-time constant defining an origin relative to another point origin.

§ 7.3 Text output

Feature	Priority	Description
`quantity` text output	1	Text output for quantity variables (number + unit)
Units text output	2	Text output for unit variables
Dimensions text output	2	Text output for dimension variables
`std::format` support	1	Custom grammar and support for all the standard formatting facilities
`std::ostream` support	2	Stream insertion operators support

Note: There is no built-in support to output `quantity_point` as text.

§ 7.4 Text input

As long as the C++ Standard doesn't provide a generic facility to parse localized text, we do not plan to propose any support for doing that for physical quantities and their units.

§ 7.5 Systems of quantities

Feature	Priority	Description
The most important ISQ quantities	1	Specification of the most commonly used ISQ quantities
ISQ 3-6	1	Specification of the ISQ quantities specified in ISO/IEC 80000 parts 3 - 6 (Space and time, Mechanics, Thermodynamics, Electromagnetism)
ISQ 7-12	3	Specification of the ISQ quantities specified in ISO 80000 parts 7 - 12 (Light and radiation, Acoustics, Physical chemistry and molecular physics, Atomic and nuclear physics, Characteristic numbers, Condensed matter physics)
ISQ 13	1	Specification of the ISQ quantities specified in IEC 80000-13 (Information science and technology)
Angular	1	Strong angular quantities
Angular ISQ	3	Changes to the ISQ to support strong angular quantities

§ 7.6 Systems of units

Feature	Priority	Description
SI	1	All the units, prefixes, and symbols (including prefixed versions) of the SI
IEC 80000-13	1	All the units, prefixes, and symbols (including prefixed versions) of IEC 80000-13
International	1	International yard and pound units (common to Imperial and USC systems)
Angular	1	Strong angular quantities and units system
Imperial	2	Imperial system-specific units
USC system	2	United States Customary system-specific units
CGS system	2	Centimetre-Gram-Second system
IAU system	3	International Astronomical Union units system

§ 7.7 Utilities

Feature	Priority	Description
`std::chrono` support	2	Customization points that enable support for `std::chrono_duration` and `std::chrono_time_point` and other external libraries
Math	2	Common mathematical functions on quantities
Random	3	Random number generators of quantities

§ 7.8 External dependencies

The features in this chapter are heavily used in the library, but are not domain-specific. Having them standardized (instead of left as exposition-only) could not only improve the specification of this library, but also could serve as an important building block for tools in other domains that we can get in the future from other authors.

Feature	Priority	Description
Number concepts	1	Concepts for vector- and point-space numbers
`fixed_string`	1	String-like structural type (can be used as an NTTP)
Compile-time prime numbers	1	Compile-time facilities to break any integral value to a product of prime numbers and their powers
Standardized type and NTTP lists	2	Common library providing algorithms to handle type and NTTP lists

§ 8 Plan for standardization

Having physical quantities and units support in C++ would be extremely useful for many C++ developers, and ideally, we should ship it in C++29. We believe that it can be done, and we propose a plan to get there. The plan below is, of course, not an irrevocable commitment. If things get delayed or controversial for whatever reason, physical quantities and units (or some parts of it) will miss the train and we will have to wait three more years. However, having a clear plan approved by LEWG will help to keep the efforts on track.

Meeting	**C++ Milestones**	**Activities**
2023.3 (Kona)		Paper on motivation, scope, and plan to LEWG, SG6, and SG23 Paper about safety benefits and concerns to SG23 Papers about quantity arithmetics and number concepts to SG6
2024.1 (Tokyo)		Paper on the Basic Framework (priority 1 features) to LEWG Paper on `fixed_string` to SG16 Paper about text output to SG16 Paper about math utilities to SG6 Paper about prime numbers to SG6
2024.2 (St. Louis)		Papers on the Basic Framework (priority 2 & 3 features) to SG6 Paper about the Affine Space to LEWG Move quantity arithmetics and number concepts papers to LEWG
2024.3 (Wrocław)		Paper of type and NTTP lists to LEWG Papers on systems to SG6 Paper about `std::chrono` support to LEWG Move text output and `fixed_string` papers to LEWG Move math utilities and prime numbers papers to LEWG
2025.1	Last meeting for LEWG review of new C++26 features	Move Basic Framework (priority 2 & 3 features) to LEWG
2025.2	C++26 CD finalized	
2025.3		Move papers on systems to LEWG

Meeting	C++ Milestones	Activities
2026.1	C++26 DIS finalized	
2026.2	C++29 WP opens	Wording for Basic Framework (priority 1 features) and number concepts to LWG
2026.3		Wording for Basic Framework (priority 2 features), the Affine Space, `std::chrono` support, the text output, and `fixed_string` to LWG
2027.1		Wording for math utilities, prime numbers, systems, and type and NTTP lists to LWG
2027.2		
2027.3		
2028.1	Last meeting for LEWG review of new C++29 features	Finalize wording review in LWG and merge into C++29 WP
2028.2	C++29 CS finalized	Resolve any outstanding design/wording issues
2028.3		Resolve NB comments
2029.1	C++29 DIS finalized	Resolve NB comments

§ 9 Acknowledgements

Special thanks and recognition goes to Epam Systems for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

We would also like to thank Peter Sommerlad for providing valuable feedback that helped us shape the final version of this document.

§ 10 References

[Au] The Au Units library.
https://aurora-opensource.github.io/au

[Clarence] Steve Chawkins. Mismeasure for Measure.
https://www.latimes.com/archives/la-xpm-2001-feb-09-me-23253-story.html

[Columbus] Christopher Columbus.
https://en.wikipedia.org/wiki/Christopher_Columbus
[Disney] Cause of the Space Mountain Incident Determined at Tokyo Disneyland Park.
https://web.archive.org/web/20040209033827/http://www.olc.co.jp/news/20040121_01en.html
[Flight 6316] Korean Air Flight 6316 MD-11, Shanghai, China - April 15, 1999.
https://web.archive.org/web/20210917190721/https://www.ntsb.gov/news/press-releases/Pages/Korean_Air_Flight_6316_MD-11_Shanghai_China_-_April_15_1999.aspx
[Gimli Glider] Gimli Glider.
https://en.wikipedia.org/wiki/Gimli_Glider
[Hochrheinbrücke] An embarrassing discovery during the construction of a bridge.
https://www.normaalamsterdamspeil.nl/wp-content/uploads/2015/03/website_bridge.pdf
[ISO/IEC Guide 99] ISO/IEC Guide 99: International vocabulary of metrology — Basic and general concepts and associated terms (VIM).
https://www.iso.org/obp/ui#iso:std:iso-iec:guide:99
[JCGM 200:2012] International vocabulary of metrology - Basic and general concepts and associated terms (VIM) (JCGM 200:2012, 3rd edition).
https://jcgm.bipm.org/vim/en
[LK8000] LK8000 - Tactical Flight Computer.
http://lk8000.it
[Mars Orbiter] Mars Climate Orbiter.
https://en.wikipedia.org/wiki/Mars_Climate_Orbiter
[Medication dose errors] Alma Mulac, Ellen Hagesaether, and Anne Gerd Granas. Medication dose calculation errors and other numeracy mishaps in hospitals: Analysis of the nature and enablers of incident reports.
https://onlinelibrary.wiley.com/doi/10.1111/jan.15072
[MISRA C++] MISRA C++:2008 Guidelines for the use of the C++ language in critical systems.
https://misra.org.uk/misra-c-plus-plus/
[mp-units] mp-units - A Physical Quantities and Units library for C++.
https://mpusz.github.io/mp-units
[nholthaus/units] UNITS - A compile-time, header-only, dimensional analysis and unit conversion library built on c++14 with no dependencies.
https://github.com/nholthaus/units
[P1930R0] Vincent Reverdy. 2019-10-07. Towards a standard unit systems library.
https://wg21.link/p1930r0
[P1935R2] Mateusz Pusz. 2020-01-13. A C++ Approach to Physical Units.
https://wg21.link/p1935r2
[P2981] Mateusz Pusz, Dominik Berner, and Johel Ernesto Guerrero Peña. Improving our safety with a physical quantities and units library.
https://wg21.link/p2981
[P2982] Chip Hogg and Mateusz Pusz. `std::quantity` as a numeric type.
https://wg21.link/p2982

[SI] SI Brochure: The International System of Units (SI).
https://www.bipm.org/en/publications/si-brochure
[SI library] SI - Type safety for physical units".
https://si.dominikberner.ch/doc
[Stonehenge] Tim Robey. Tiny stones, giant laughs: the story behind Spinal Tap's Stonehenge.
https://www.telegraph.co.uk/films/2020/05/01/tiny-stones-giant-laughs-story-behind-spinal-taps-stonehenge
[Vasa] Rhitu Chatterjee and Lisa Mullins. New Clues Emerge in Centuries-Old Swedish Shipwreck.
https://theworld.org/stories/2012-02-23/new-clues-emerge-centuries-old-swedish-shipwreck
[Wild Rice] Manufacturers, exporters think metric.
https://www.bizjournals.com/eastbay/stories/2001/07/09/focus3.html